Composite

Season: 2 · Structural · Category: GoF · Level: Beginner · Build with Waqar

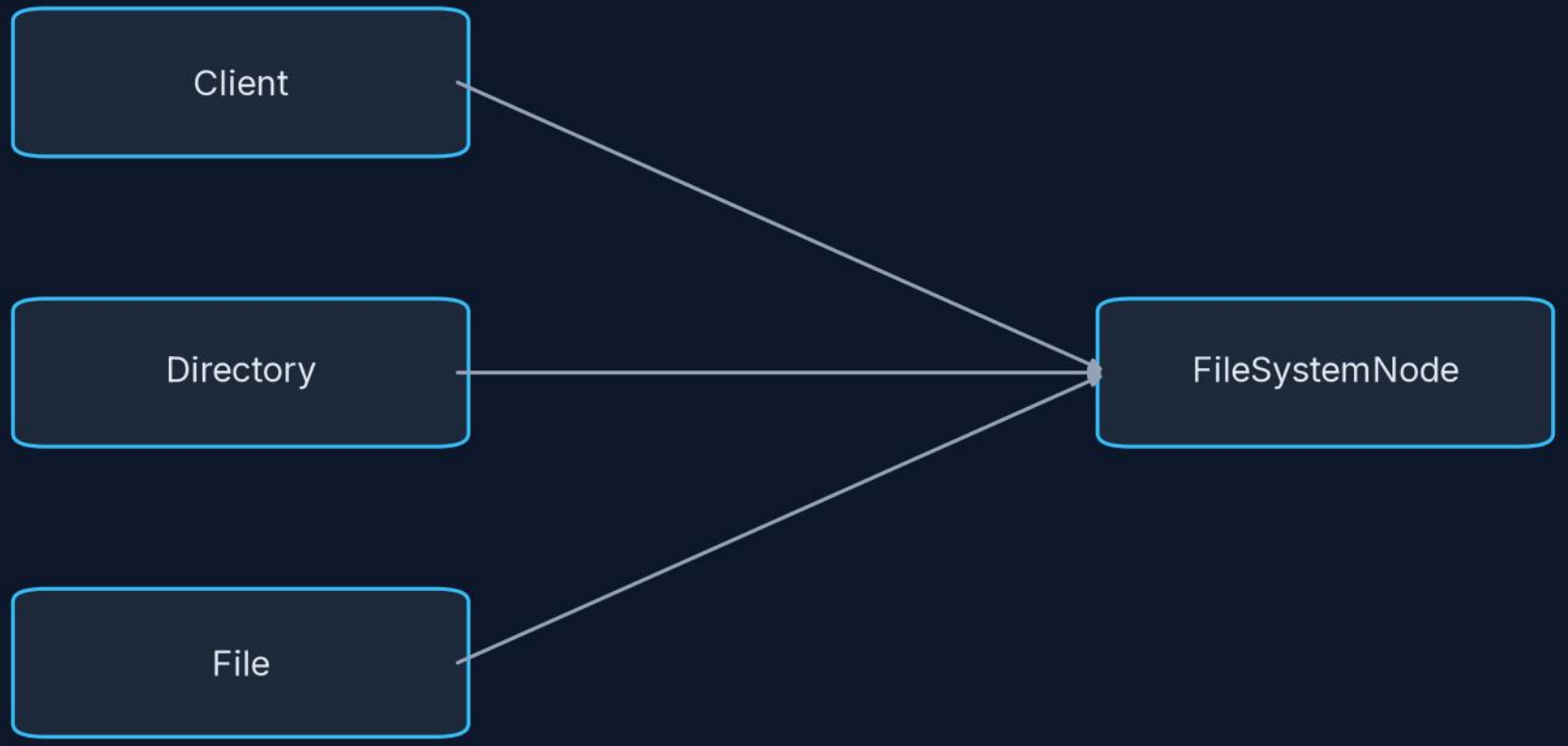
Today's Hook

- Today: Composite as a StreamForge product force — not trivia.
- Runnable Demo on JDK 25 with captured stdout.
- Classic encoding → Java 25 / Zazr modernization → trade-offs.
- CapCut slides + full spoken script included.

The Problem

- Naive local fixes: more ifs, statics, nulls, or threads.
- Illegal states still compile; tests become order-dependent.
- Design reviews lack a shared name for the force.
- Interviews punish memorization without judgment.

Structure



Composite structure

Real-World Scenario

- StreamForge creator platform is the running metaphor.
- Same names appear in naive code, refactor, and Demo output.
- Stay in one story for the full 10–12 minutes.
- Product language first; pattern vocabulary second.

Naive Code

```
// Naive sketch for Composite
public class Naive {
  // duplicated logic / leaked details / unsafe sharing
  Object doIt(Object... args) {
    if (args == null) return null;
    return args[0];
  }
}
```

Why Naive Hurts

- Coupling spreads across controllers and infrastructure.
- Tests need sleeps or static reset hooks.
- New variants require shotgun edits.
- On-call cannot reason about failure modes.

Implementation

```java
// file: src/main/java/com/buildwithwaqar/patterns/gof/structural/composite/DirectoryComposite.java
package com.buildwithwaqar.patterns.gof.structural.composite;

import java.util.ArrayList;
import java.util.List;

sealed interface FileSystemNode permits FileLeaf, DirectoryComposite {
    String name();
    int size();
}

record FileLeaf(String name, int size) implements FileSystemNode {}

public final class DirectoryComposite implements FileSystemNode {
    private final String name;
    private final List<FileSystemNode> children = new ArrayList<>();

    public DirectoryComposite(String name) { this.name = name; }

    public DirectoryComposite add(FileSystemNode node) {
        children.add(node);
        return this;
    }
```

Implementation (2)

```
    @Override public String name() { return name; }

    @Override public int size() {
        return children.stream().mapToInt(FileSystemNode::size).sum();
    }
}
```

Demo.main

```java
package com.buildwithwaqar.patterns.gof.structural.composite;
public final class Demo {
  public static void main(String[] a) {
    System.out.println("=== Composite walkthrough ===");
    var root = new DirectoryComposite("root")
      .add(new FileLeaf("readme.md", 120))
      .add(new DirectoryComposite("src").add(new FileLeaf("App.java", 800)).add(new FileLeaf("Demo.java", 400)));
    System.out.println("rootSize=" + root.size());
    System.out.println("rootName=" + root.name());
  }
}
```

Walkthrough Output

```
=== Composite walkthrough ===
rootSize=1320
rootName=root
```

Step-by-Step Trace

- Each stdout line maps to a refactor stage.
- Re-run Demo locally on JDK 25 to verify.
- Differences usually mean a dirty tree — pull main.
- Pause the video and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction for Composite
// records + sealed + switch
// Option/Either/Validation instead of null
// virtual threads for blocking fan-out
// inject interfaces at the edge
```

Trade-offs

- Clarity vs ceremony — do not spawn five types for two lines.
- Closed sealed worlds vs open SPI plugins.
- Teaching demos omit production idempotency/metrics.
- Prefer simpler language features when they erase the force.

Related Patterns

- Contrast with the neighbor pattern (force difference).
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Force → naive pain → Composite → Demo stdout.
- Modernize with Java 25 / Zazr when it shrinks code.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run Demo.